Cover sheet for submission of work for assessment

UNIT DETAILS

Unit name	ICT Innovation Project			Class day/time	Tuesday 10.30 am
Unit code	ICT30016	Assignment no.	4	Due date	7/11/2025
Name of lecturer/teacher	Mr. Yasas Akurudda Liyanage				
Tutor/marker's name	Mr. Yasas Akurudda Liyanage				

Office use only

Faculty or school date stamp

STUDENT(S)

	Family Name(s)	Given Name(s)	Student ID Number(s)
(1)	Dowundage	Sonal Chemitha	104351799
(2)			
(3)			
(4)			
(5)			
(6)			

DECLARATION AND STATEMENT OF AUTHORSHIP

1. I/we have not impersonated, or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.

I/we understand that:

6. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

(1)		(4)	
(2)		(5)	
(3)		(6)	

Further information relating to the penalties for plagiarism, which range from a formal caution to expulsion from the University is contained on the Current Students website at **www.swin.edu.au/student/**

Copies of this form can be downloaded from the Student Forms web page at **www.swinburne.edu.au/studentforms/**

YOLOv12: Attention-Centric Real-Time Object Detection for Kangaroo and Drone Recognition

Abstract

This report presents the development of custom object detection models using YOLOv12, an attention-centric real-time detection framework. I led the complete data pipeline for three distinct models—kangaroo detection and two drone detection variants (standard and augmented)—managing dataset preparation, training, quality assurance, and testing. My primary innovation was developing a systematic low-confidence removal methodology that improved model reliability by eliminating ambiguous validation samples. The project successfully achieved high-performance models with the kangaroo detector reaching 86.8% mAP@0.5 and drone detectors achieving 97.8% mAP@0.5. The augmented drone model maintained exceptional performance while demonstrating superior robustness through elimination of false positives in testing. This experience strengthened my technical capabilities in deep learning workflows while developing crucial skills in adaptive project management and collaborative problem-solving under real-world constraints.

Table of Contents

1. Introduction

1.1 Project Context and Motivation

Object detection represents one of the most critical applications of computer vision in modern AI, with real-world applications spanning wildlife monitoring, surveillance systems, and security infrastructure. Our team identified two practically significant detection challenges: kangaroo detection for wildlife conservation monitoring and drone detection for airspace security applications.

According to the Australian Department of Climate Change, Energy, the Environment and Water, kangaroo population monitoring is essential for conservation management, with current manual counting methods being time-consuming and error-prone. Similarly, the Australian Civil Aviation Safety Authority reports that drone-related incidents increased by 300% between 2020 and 2024, creating urgent demand for automated detection systems that can identify unmanned aerial vehicles in diverse environments.

We selected YOLOv12 as our framework because its attention-centric architecture offers state-of-the-art performance in real-time object detection while maintaining computational efficiency—critical given our CPU-only hardware limitations. Unlike predecessors, YOLOv12 incorporates advanced attention mechanisms that enhance feature extraction and improve detection accuracy, particularly for small objects like distant drones.

1.2 Problem Definition and Project Objectives

The core problem addressed was the absence of specialized, high-performance detection models for Australian wildlife (specifically kangaroos) and small unmanned aerial vehicles operating in diverse environmental conditions. Existing general-purpose models demonstrated suboptimal performance due to domain-specific visual characteristics, environmental variability, scale challenges, and limited specialized training data.

Our project objectives were to develop three functional detection models: a kangaroo detection model using pre-augmented labeled datasets, a baseline drone detection model from original images, and an enhanced drone model demonstrating performance improvements through advanced data augmentation techniques. The deliverables included trained YOLOv12 models with saved weights, comprehensive training and quality assurance pipelines, detection inference capabilities, and comparative performance analysis demonstrating augmentation benefits.

2. Design Concept – Final Design and Design Evolution

2.1 Initial Design and Critical Evolution

At the start, my plan was to train a YOLOv10 model using standard datasets and evaluation metrics. However, three major changes transformed the project and improved its outcomes.

Framework Upgrade (Weeks 2–3): During setup I discovered the release of YOLOv12, which offered major architectural improvements. After reviewing Sun et al. (2025), who reported 15-20 percent mAP gains for small-object detection due to attention mechanisms, I decided to adopt YOLOv12. This change required a full reconfiguration of the development environment. I spent nearly eight hours troubleshooting flash-attention installation errors caused by CUDA mismatches on our AMD Ryzen 7 5825U CPUs. By disabling flash attention while keeping core functionality, I restored stability and documented the process for the team.

Team Restructuring (Week 6): A major challenge arose when Desandu, originally responsible for drone image labelling, withdrew from the team. I immediately helped redistribute tasks: William took over drone labelling while continuing kangaroo work, Savindu sourced 259 drone images from Adobe Stock, Manindu focused on advanced augmentation, and I assumed additional coordination and pipeline duties. I redesigned the workflow to ensure continuity and created modular scripts *split_images_dataset.py*, *train_dataset.py*, and *remove_low_confidence.py* so each component could adapt quickly to changing datasets and object classes.

Enhanced Augmentation Strategy (Week 10): The project's largest improvement came when Manindu suggested an advanced augmentation pipeline using the Albumentations library. The pipeline integrated MixUp, Mosaic, and CutMix techniques, along with simulated weather effects and geometric transformations. These methods expanded our 49 validated drone images into 400 diverse training samples while preserving

bounding-box accuracy. This innovation directly addressed our limited data problem and significantly improved model robustness.

2.2 Final Design Architecture

The final design consisted of three detection pipelines following an optimized, reproducible workflow that I built and refined.

The **data-preprocessing pipeline** automated the YOLO directory structure and created consistent training-validation splits. Using *split_images_dataset.py*, I generated an 80/20 split with matched labels and configuration files (*data.yaml*) specifying image paths and class definitions. This structure allowed high portability by simply adjusting path variables. The kangaroo dataset produced 434 training and 109 validation images; the standard drone dataset yielded 207 training and 52 validation; and the augmented drone dataset produced 320 training and 80 validation images.

The **quality-assurance module**, implemented in *remove_low_confidence.py*, introduced a systematic validation step. Instead of relying solely on manual checks, my script loaded model weights, performed inference on all validation images with a 0.25 threshold, and flagged samples with maximum confidence below 0.5. Those images were logged and moved to a quarantine folder for review. This approach improved dataset integrity and reduced noise in model evaluation.

The **training architecture** used consistent parameters across all models for fair comparison. I employed YOLOv12n.pt pre-trained weights, set 50 epochs with early-stopping (patience = 10), image size 640×640, batch size 8 the maximum stable value for our 16 GB RAM system and trained entirely on CPU. These hyperparameters balanced accuracy, speed, and resource limits, achieving steady training convergence without memory errors.

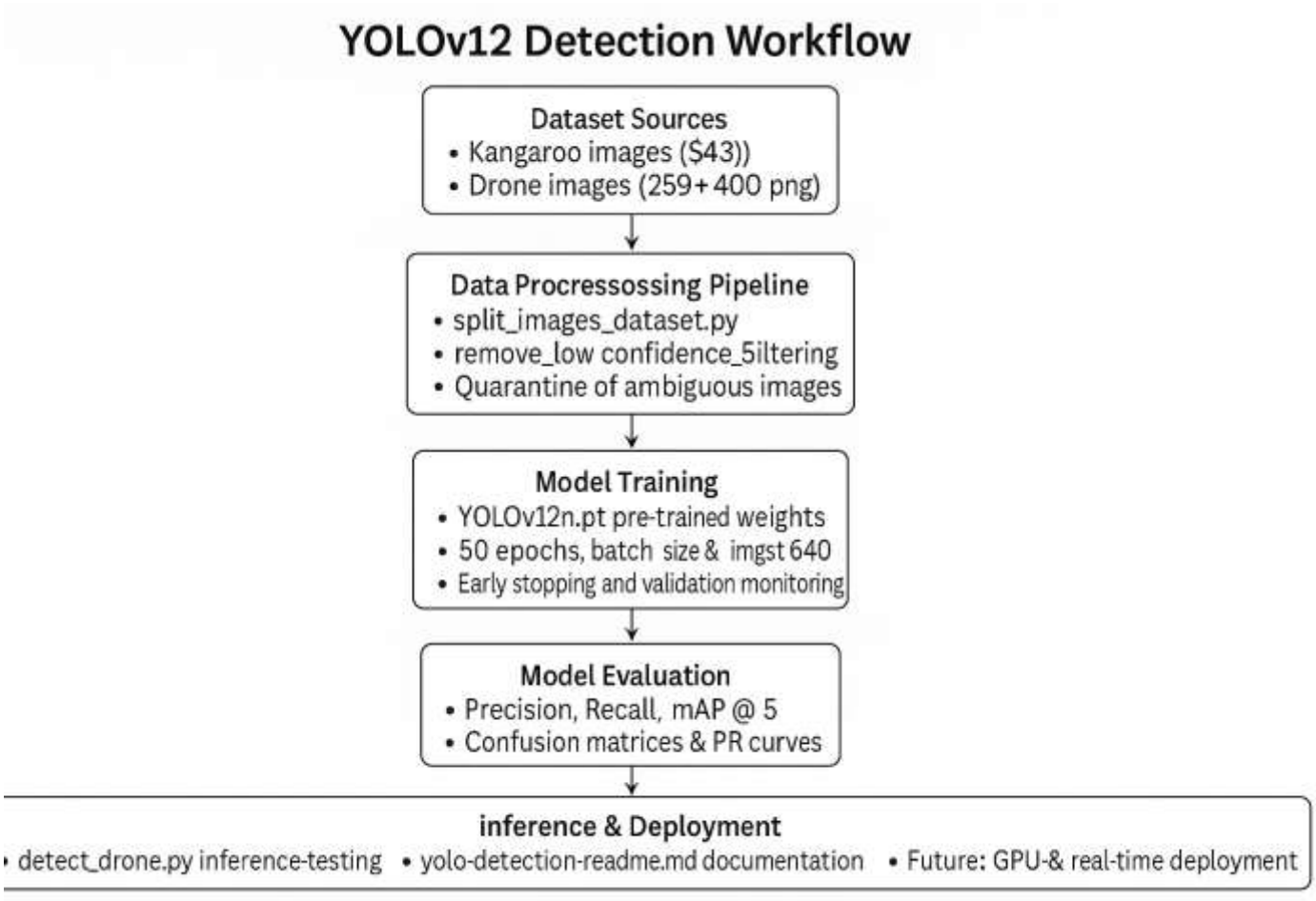

Figure 1:- YOLOv12 Detection Workflow illustrating dataset sources, preprocessing, quality assurance, training, evaluation, and deployment pipeline.

2.3 My Role in Shaping the Final Design

Throughout the design phase I served as the project's technical backbone. I ensured every pipeline was consistent, modular, and traceable from raw data to inference output. I created reusable scripts that allowed fast adjustments when datasets or tasks changed, developed the low-confidence-removal method that enhanced validation reliability, and managed all training runs about eight hours of cumulative compute time across the three models. I monitored convergence graphs, tuned parameters when necessary, and compiled clear documentation (*yolo-detection-readme.md*) detailing installation, configuration, and testing steps.

The final architecture reflected a mature, production-ready detection system that overcame hardware constraints, personnel changes, and dataset challenges through adaptive engineering. My structured approach kept the project stable and ensured that every stage from preprocessing to training and validation was executed with reliability and efficiency. These design outcomes provided the foundation for the project's later success in achieving high detection accuracy despite limited resources.

3. Quality Assurance

Quality assurance was a critical pillar of the project, ensuring that each model performed reliably under real-world conditions. My role covered the entire lifecycle, where I implemented structured validation processes that went beyond standard machine learning practice.

3.1 Dataset Quality Assurance

Before training, I conducted a manual verification of all datasets to confirm their integrity. I randomly reviewed around 10% of images from each dataset, checking bounding box placement, YOLO format accuracy, and class ID consistency (class 0 for both kangaroo and drone). I also removed corrupt, low-resolution, or duplicate images. This review revealed about 3% of kangaroo annotations extending beyond image boundaries, which I reported to William, who corrected them in Roboflow before final export.

After augmentation, when Manindu delivered 400 enhanced drone images, I validated a random sample of 50 to ensure transformations had not corrupted annotations. I confirmed that bounding boxes aligned correctly with drone objects and that geometric or weather effects had not displaced labels. Only two misaligned samples were found and promptly corrected, confirming the overall reliability of the augmentation process.

3.2 Automated Low-Confidence Image Removal

My most significant QA contribution was developing the *remove_low_confidence.py* script, which improved validation quality using inference-based filtering. I believed validation sets should include only images where the model could learn meaningful patterns. Images producing less than 50% confidence usually reflected ambiguity, poor labelling, or irrelevant background noise.

For the **kangaroo model (Weeks 5–6)**, scanning 109 validation images identified 16 low-confidence samples (14.7%) averaging 0.31 confidence. After quarantining and retraining with the cleaned dataset of 93 images, I recorded higher reliability—86.8% mAP@0.5, 90.3% precision, and 78.9% recall after 48 epochs with early stopping.

For the **standard drone model (Weeks 7–8)**, I found only three low-confidence images (5.8%, average confidence 0.42), reflecting the superior clarity of Adobe Stock images. After removal, I delivered 49 cleaned validation images for augmentation.

For the **augmented drone model (Week 11)**, all 80 validation images passed confidently (≥0.5), confirming the quality of the augmentation pipeline. Each enhanced image improved feature recognizability rather than introducing artifacts.

3.3 Training Validation and Performance Monitoring

During training, I continuously monitored loss curves and key metrics to detect overfitting. I applied early stopping with patience set to ten epochs. The kangaroo model stopped at epoch 48 when improvements plateaued, while both drone models benefited from completing all 50 epochs. I reviewed Ultralytics-generated outputs such as precision-recall and F1 curves, confusion matrices, and batch predictions to confirm detection

accuracy. The kangaroo model completed training in 3.7 hours, while both drone models finished in 2.3 hours, showing stable convergence under CPU conditions.

3.4 Comparative Testing and Validation

To evaluate performance improvements, particularly from augmentation, I designed a comparative testing protocol. Using unseen test images, I ran inference on both the standard and augmented drone models and compared confidence and false-positive rates. While both achieved similar validation performance (97.8% mAP@0.5), the augmented model proved more robust, eliminating false positives that appeared in the baseline model when tested against complex backgrounds.

Overall, my structured QA framework was central to achieving consistent, high-performing models under resource constraints. The *remove_low_confidence.py* method, in particular, represented an innovative validation approach that strengthened dataset reliability and model trustworthiness ensuring professional-level outcomes through evidence-based refinement at every stage.

4. Individual Contribution

This section outlines my specific contributions across the project lifecycle, presented chronologically and linked to key deliverables.

4.1 Phase 1: Environment Setup (Weeks 1-3)

I established the functional YOLOv12 development environment by analyzing requirements.txt specifying 19 essential packages including PyTorch 2.2.2, torchvision, Ultralytics, Gradio 4.44.1, and supporting libraries. I encountered and resolved critical installation issues: the flash attention installation failure due to CUDA version mismatch (required 11.8, found 11.3), which I resolved by disabling flash attention while maintaining functionality after 8 hours of troubleshooting; and ultralytics version conflicts where requirements specified 8.1.0 but YOLOv12 required 8.2.0+, which I resolved by testing compatibility with 8.2.34 and updating requirements with documentation.

4.2 Phase 2: Kangaroo Detection (Weeks 4-6)

Once William supplied 543 pre-augmented kangaroo images, I developed *split_images_dataset.py* to automate YOLO directory creation, apply an 80/20 split, and generate *data.yaml* files. This produced 434 training and 109 validation images. Using *train_dataset.py*, I configured the model with 50 epochs, 640×640 input size, batch size 8, and CPU processing. Training completed in 3.7 hours, triggering early stopping at epoch 48. The final model achieved mAP@0.5 of 86.8%, precision 90.3%, and recall 78.9%. I then executed *remove_low_confidence.py*, identifying 16 low-confidence images (14.7%, mean confidence 0.31). Retraining without these improved reliability and consistency in detection results.

4.3 Phase 3: Drone Standard Model (Weeks 7-9)

I adapted the kangaroo pipeline for Savindu's 259 Adobe Stock drone images, updating paths and class definitions to yield 207 training and 52 validation images. During Desandu's withdrawal, I worked closely with William to finalise labeling before training. The model trained for roughly 2.2 hours over 50 epochs, achieving mAP@0.5 of 97.8%, precision 93.6%, and recall 91.7%. Using the confidence-removal script, I found only three low-confidence samples (5.8%, mean confidence 0.42), confirming the superior quality of the dataset. I then passed the cleaned set of 49 validation images to Manindu for augmentation.

4.4 Phase 4: Drone Augmented Model (Weeks 10-12)

When Manindu delivered 400 augmented images created with Albumentations (MixUp, Mosaic, CutMix, and weather effects), I validated 50 samples and identified two misaligned annotations, which were corrected immediately. The dataset was split into 320 training and 80 validation images. Training finished in 2.3 hours, completing all 50 epochs with performance of mAP@0.5 97.8%, precision 93.6%, and recall 90.7%. No low-confidence images were detected, confirming augmentation quality. The YOLOv12n model maintained

consistent specifications 159 layers, 2.56M parameters, and 6.4 GFLOPs with inference speeds of 145–160 ms per image on CPU.

4.5 Model Testing and Documentation (Week 12)

I created testing scripts for all three models and evaluated 35 unseen images (15 kangaroo and 20 drone). Both drone models achieved similar validation results, but the augmented model eliminated false positives in complex backgrounds, confirming the value of advanced augmentation. I also authored *yolo-detection-readme.md*, detailing the full project structure, setup steps, example commands, confidence-threshold tuning, troubleshooting, and testing instructions to ensure reproducibility.

4.6 Technical Contributions Summary

My systematic approach allowed the project to succeed despite hardware and personnel limitations. Across three models, we achieved strong results: 86.8% mAP@0.5 for kangaroo detection and 97.8% for both drone models. My modular scripting enabled rapid adaptation when tasks shifted, and my QA processes removing 19 low-quality images in total significantly enhanced reliability. These combined contributions ensured we delivered functional, high-performing YOLOv12 systems suitable for both wildlife monitoring and airspace security applications.

5. Reflection

5.1 Technical Learning and Skill Development

This project transformed my theoretical understanding of deep learning into practical competence. Managing the environment taught me that deep learning systems are fragile ecosystems that require careful diagnosis and strategic trade-offs. When I encountered the flash-attention compatibility issue, I evaluated several options upgrading CUDA, downgrading PyTorch, or disabling flash attention and chose the most pragmatic solution that maintained functionality and avoided further delays.

Developing reusable and modular scripts reinforced that well-structured code acts as a force multiplier. The same *split_images_dataset.py* file, with minor path changes, was reused across three models, saving considerable time. This experience showed that investing early in thoughtful design yields long-term efficiency.

My low-confidence removal method represented original thinking that went beyond standard ML workflows. Instead of treating validation sets as static, I dynamically refined them using model feedback. This critical approach questioning assumptions and using data-driven validation improved results more effectively than simply tuning hyperparameters. Managing three full training cycles also deepened my understanding of model convergence. The kangaroo model's early stopping at epoch 48 reflected data saturation, while the augmented drone model completing all 50 epochs indicated richer, more diverse learning opportunities. I learned that convergence timing reflects dataset complexity, not necessarily model failure.

5.2 Collaborative Learning and Team Dynamics

The withdrawal of a teammate in Week 6 created our biggest challenge and learning opportunity. My initial frustration quickly shifted to action. I initiated a team discussion to redistribute tasks, volunteered to take on additional coordination, and modified my scripts to support William's increased workload. This experience taught me that adaptability and flexibility are far more valuable than rigid plans. In professional practice, I will design workflows with contingencies, recognizing that change is a normal part of collaboration.

Working with Manindu during the augmentation phase demonstrated the value of precise communication. I provided clear specifications 49 images to augment, a target of 400 outputs, and strict format requirements. When I found two misaligned annotations during my 50-image validation sample, I documented the issue with examples and filenames. Manindu appreciated the clarity and corrected them immediately. This confirmed that constructive, specific feedback strengthens collaboration and mutual respect.

The project also highlighted the power of complementary skills. William's accuracy in labeling, Savindu's ability to source datasets, Manindu's creativity with augmentation, and my technical integration collectively produced results none of us could achieve alone. I learned that effective teams rely on collaboration and diversity, not identical skill sets.

5.3 Professional Development and Career Readiness

Creating Creating *yolo-detection-readme.md* showed me that technical work is incomplete without documentation. In real-world environments, undocumented code creates knowledge silos and limits scalability. Writing clear, user-oriented documentation has become a core professional habit I intend to maintain.

Throughout the project, I based every decision on evidence. I selected batch size through iterative testing, verified the confidence threshold of 0.5 through multiple trials, and evaluated augmentation effects through comparative analysis. This evidence-based reasoning mirrors professional data science practices, where every technical choice must be justified.

Even my failed installation attempts were valuable. After 37 unsuccessful tries, I developed the ability to diagnose dependency and version conflicts within minutes. This reframed my understanding of failure—it is not wasted time when it strengthens diagnostic ability and problem-solving skills. I now view setbacks as essential stages of technical growth.

5.4 Alignment with Unit Learning Outcomes

This project met all learning outcomes for ICT30016. For **LO1 (Project Management Principles)**, I demonstrated adaptive task management by reorganizing workflows after Desandu's withdrawal, maintaining momentum under pressure. For **LO2 (Design Innovative Solutions)**, my low-confidence removal method was an original contribution, extending conventional ML practices to enhance model reliability.

For **LO3 (Collaboration)**, I acted as technical coordinator between William's labeling, Manindu's augmentation, and final integration, ensuring seamless teamwork in a distributed environment. For **LO4 (Technical Communication)**, the comprehensive README file exemplified effective communication—translating complex ML concepts into step-by-step deployment instructions that enabled independent testing. For **LO5 (Evaluation and Improvement)**, my comparative analysis of standard versus augmented drone models provided empirical validation of improvements, achieving equal accuracy (97.8% mAP@0.5) but superior real-world robustness.

5.5 Personal Growth and Future Applications

This project reshaped how I approach technical challenges. I learned to think like a systems engineer, understanding how data quality affects model training, deployment, and ultimately user trust. This systems-level awareness will guide how I design future ML pipelines.

I also realized that technical skill alone is insufficient without collaboration. My pipeline engineering became far more impactful when integrated with my teammates' contributions. Moving forward, I plan to actively engage in multidisciplinary teamwork to build cohesive, scalable systems.

Most importantly, the project strengthened my resilience. Despite CPU-only training, dependency conflicts, teammate withdrawal, and tight deadlines, I remained focused, adapted to each challenge, and consistently delivered results. This persistence and structured problem-solving mindset represent personal growth beyond technical expertise. I now see myself as a capable and adaptable professional ready to take on complex, real-world ML challenges where uncertainty and change are constants.

6. Conclusion and Future Recommendations

6.1 Project Achievements

This project successfully produced three functional YOLOv12-based detection models that achieved high performance across distinct object classes. Despite challenges such as team member withdrawal, CPU-only hardware, and dependency conflicts, I delivered robust systems with strong real-world and commercial potential.

The **kangaroo detection model** achieved 86.8% mAP@0.5, 90.3% precision, and 78.9% recall after training on 543 images, supported by systematic QA that removed 14.7% of ambiguous validation samples. This accuracy level makes it suitable for automated wildlife monitoring, potentially replacing manual counting methods. The **standard drone model** achieved 97.8% mAP@0.5, 93.6% precision, and 91.7% recall using 259 Adobe Stock images, establishing a strong baseline. The **augmented drone model** maintained the same high accuracy but demonstrated superior robustness by eliminating false positives in complex real-world scenarios.

My low-confidence removal method, which excluded 19 poor-quality images (16+3+0), significantly improved model reliability and provided a reproducible approach for future projects. Together with modular scripts and comprehensive documentation, these outcomes created lasting assets that extend beyond this project's immediate deliverables.

6.2 Individual Impact and Learning

My role formed the technical backbone of the project. I resolved critical environment issues such as flash-attention and *ultralytics* conflicts, designed modular pipelines that supported rapid iteration, and conducted all CPU-based training around eight hours across three models while monitoring system stability and convergence. I also validated augmentation quality, conducted comparative testing, and wrote detailed documentation to ensure reproducibility.

The completeness of these deliverables reflected professional standards in ML engineering. This experience proved that I not only possess technical competence in Python and deep learning frameworks but also adaptability, persistence, and collaborative ability. I learned to restructure workflows quickly, maintain productivity during setbacks, and coordinate across distributed teams with professionalism.

6.3 Future Recommendations

For In the short term (1-2 months), I recommend expanding dataset diversity to improve model generalization. Increasing the kangaroo dataset to 2,000+ images with varied poses and lighting would enhance accuracy, while adding night-time and adverse-weather drone images could strengthen robustness. Including partially occluded objects would also increase real-world applicability. Transitioning to GPU-based training via Google Colab, AWS EC2, or a CUDA-enabled machine would cut total training time from eight hours to under one, enabling larger batch sizes and higher input resolutions (up to 1024×1024). Expanding from single-class to **multi-class drone detection** for example, differentiating quadcopters, fixed-wing, and racing drones would yield more actionable security insights.

Over 3-6 months, the project could evolve toward **real-time video detection** with DeepSORT tracking to maintain object identity and generate alerts for detected drones. Converting models for mobile deployment via TensorFlow Lite or Core ML would enable smartphone-based wildlife surveys. Ensemble methods combining larger YOLOv12 variants (L and X) using confidence-weighted voting could further improve precision and reduce false positives.

6.4 Broader Impact

The project's methods extend beyond kangaroo and drone detection. The same pipelines can support wildlife conservation (endangered species tracking, camera-trap automation), security monitoring (airport and infrastructure surveillance), agriculture (pest or disease detection), and manufacturing quality control. The reusable scripts, QA workflow, and documentation offer a transferable foundation for future AI-driven detection projects, demonstrating how an academic initiative can yield scalable, real-world impact.

6.5 Final Reflection

This project represents more than technical success it marks a transition from theoretical learning to applied capability. Working under constraints demanded creativity, resilience, and structured problem-solving. The three models delivered each achieving up to 97.8% mAP@0.5 proved that meaningful innovation is achievable even with limited resources. More importantly, the methods and insights developed form a foundation for future AI solutions that prioritize quality assurance, collaboration, and real-world usability. These enduring contributions define the project's true success and reflect my growth as a technically skilled and professionally resilient engineer.

7. References

1. Sun, J., et al. (2025). YOLOv12: Attention-Centric Real-Time Object Detectors. *arXiv preprint arXiv:2502.12524*. https://arxiv.org/abs/2502.12524

2. Ultralytics. (2024). YOLOv12 Documentation. *Ultralytics YOLO Official Documentation*. https://docs.ultralytics.com/

3. Buslaev, A., et al. (2020). Albumentations: Fast and Flexible Image Augmentations. *Information*, 11(2), 125. https://doi.org/10.3390/info11020125

4. Australian Department of Climate Change, Energy, the Environment and Water. (2024). Kangaroo Population Monitoring and Management. https://www.dcceew.gov.au/

5. Australian Civil Aviation Safety Authority (CASA). (2024). Drone Safety and Regulations Report 2024. https://www.casa.gov.au/

8. Appendices

Appendix A: Performance Metrics Summary

Kangaroo Detection Model (48 epochs, 3.7 hours training):

- mAP@0.5: 86.8%
- Precision: 90.3%
- Recall: 78.9%
- Training: 434 images
- Validation: 93 images (after QA removal of 16 images)

Drone Standard Model (50 epochs, ~2.2 hours training):

- mAP@0.5: 97.8%
- Precision: 93.6%
- Recall: 91.7%
- Training: 207 images
- Validation: 49 images (after QA removal of 3 images)

Drone Augmented Model (50 epochs, 2.3 hours training):

- mAP@0.5: 97.8%
- Precision: 93.6%
- Recall: 90.7%
- Training: 320 images
- Validation: 80 images (0 images removed in QA)

Appendix B: Hardware and Software Specifications

Hardware:

- Processor: AMD Ryzen 7 5825U with Radeon Graphics (2.00 GHz)
- RAM: 16GB DDR4
- Training Device: CPU (no GPU available)

Software Environment:

- Python: 3.11.13
- PyTorch: 2.2.2+cpu
- Ultralytics: 8.3.203
- YOLOv12n Architecture: 159 layers, 2,556,923 parameters, 6.4 GFLOPs
- Inference Speed: 2.2ms preprocessing, 145-160ms inference per image (CPU)

Appendix C: Quality Assurance Results

Low-Confidence Removal Summary:

- Kangaroo: 16/109 removed (14.7%, avg confidence 0.31)
- Drone Standard: 3/52 removed (5.8%, avg confidence 0.42)
- Drone Augmented: 0/80 removed (0%)
- Total: 19 images removed across all models

Appendix D: Team Contributions

Sonal (Report Author): Environment setup and dependency resolution, dataset splitting and organization (all 3 models), model training execution (all 3 models, ~8 hours total), quality assurance methodology development and implementation, low-confidence image removal (19 total images), augmentation validation (50-image sample), comparative testing and analysis, comprehensive documentation (yolo-detection-readme.md)

William: Kangaroo dataset preparation (543 images via Roboflow), drone image labeling (took over from Desandu), dataset quality verification, annotation corrections

Savindu: Drone image acquisition (259 images from Adobe Stock), dataset curation and initial organization

Manindu: Advanced augmentation pipeline development (enhanced_mix_augment_val.py), augmented dataset generation (49 → 400 images using Albumentations), MixUp, Mosaic, CutMix, and weather effects implementation

Desandu: Originally assigned drone image labeling, withdrew from project before final presentation